

Universidad de Guadalajara
Centro universitario de Tonalá CUT
Ing. en ciencias computacionales



Programación de Servicios Web

Alumnos:

Osmar Daniel Campechano García

Braulio Ivan Hernández Fernández

Alan Fernando Ramos Aguilar

Miguel Angel Nicasio Medina

Profesor:

Jose Navarro

Concepto de Arquitectura Orientada a Servicios (SOA)

La Arquitectura Orientada a Servicios (SOA) es un estilo arquitectónico de integración para toda la empresa que define un método para hacer que los componentes de software sean reutilizables e interoperables a través de interfaces de servicio. Cada servicio en SOA incorpora el código y los datos necesarios para ejecutar una función empresarial completa y discreta (como comprobar el crédito de un cliente, calcular el pago de un préstamo o procesar una hipoteca).

La interfaz de servicio actúa como un contrato entre el proveedor y el consumidor del servicio. Se expone mediante protocolos de red estándar como SOAP/HTTP o RESTful HTTP (JSON/HTTP), lo que permite que las aplicaciones se comuniquen independientemente de la tecnología en la que estén desarrolladas (Java, Microsoft .NET, COBOL, software empaquetado como SAP, SaaS o código abierto).

Características Principales

- Acoplamiento flexible (Loose coupling): Las interfaces permiten llamar a los servicios con poco o ningún conocimiento de su implementación subyacente, reduciendo drásticamente las dependencias entre aplicaciones.
- Reutilización e interoperabilidad: Evita duplicar o reescribir funciones existentes en cada proyecto de desarrollo. Mediante tecnologías estándar como HTTP, SOAP y WSDL, permite la comunicación entre plataformas diversas.
- Definición mediante estándares: Las interfaces se definen habitualmente mediante lenguajes como WSDL (basado en XML), que especifica la estructura del servicio como un conjunto de puntos finales que operan sobre mensajes.
- Gobernanza y Registro: La gobernanza controla el ciclo de vida del servicio. Los servicios se publican en un registro centralizado para que los desarrolladores puedan encontrarlos y reutilizarlos en nuevos procesos.

Componentes de SOA

1. Interfaces y Contratos de Servicio: Definen el punto de acceso a la función empresarial y especifican cómo enviar o modificar datos mediante mensajes de red.
2. Bus de Servicio Empresarial (Enterprise Service Bus - ESB): Es un patrón arquitectónico clave donde un componente centralizado realiza integraciones entre aplicaciones. Se encarga de:
 - Transformación de modelos de datos.
 - Conectividad, mensajería y enrutamiento.
 - Conversión de protocolos de comunicación y composición de solicitudes. (Aunque es posible implementar SOA sin un ESB, esto derivaría en conexiones punto a punto complejas y difíciles de mantener).
3. Registro de Servicios: Catálogo central donde se publican los servicios para permitir su búsqueda y reutilización por parte de los desarrolladores.
4. Sistemas de Registro o Heredados (Legacy): Funciones de aplicaciones existentes o mainframes que se exponen a través de interfaces de servicio para su aprovechamiento.

Ventajas de SOA

- Mayor agilidad empresarial y tiempo de comercialización más rápido: Ensamblar aplicaciones a partir de bloques reutilizables permite a los desarrolladores enfocar su tiempo en entregar mejoras en lugar de gestionar integraciones desde cero.
- Aprovechamiento de funciones heredadas: Permite tomar capacidades "bloqueadas" en plataformas antiguas (como mainframes) y extenderlas hacia nuevos entornos, como aplicaciones web.

- Eliminación de integraciones punto a punto: Sustituye las conexiones personalizadas e individuales entre sistemas por un modelo de servicio estandarizado.
- Mejor colaboración entre el negocio y TI: Los servicios se definen en términos del negocio (ej. "generar cotización de seguros"), facilitando que los analistas y desarrolladores diseñen y adapten flujos de trabajo en conjunto.

Aplicaciones en el Desarrollo de Software

- Modernización e integración de sistemas: Exposición de datos y funciones financieras en mainframes hacia aplicaciones web para que los clientes accedan a sus trámites de forma directa.
- Casos de uso empresariales en la industria:
 - Delaware Electric: Integró sistemas aislados mediante SOA para aumentar la eficiencia operativa y mantener la estabilidad financiera.
 - Cisco: Implementó SOA para ofrecer una experiencia de pedidos uniforme en todos sus productos y canales, exponiendo los procesos de compra como servicios.
 - Independence Blue Cross (IBC): Garantizó que consultorios médicos, agentes y usuarios web trabajaran con una única fuente de verdad sobre los datos de los pacientes.
- Desarrollo de Servicios Web Interoperables: Creación de servicios basados en mensajes o RPC (Remote Procedure Call) utilizando estándares de comunicación como JAX-WS sobre SOAP y HTTP.

Descripción de las cuatro capas principales:

Para el proyecto utilizaremos un diseño que combina los principios de SOA con una arquitectura de cuatro capas.

Es importante aclarar que SOA no exige una única estructura universal de cuatro capas. Esta organización es una propuesta para cumplir los requisitos del curso y facilitar la separación de responsabilidades.

Arquitectura de cuatro capas

1. Capa de Presentación: Página web · Formularios · Panel de administrador

2. Capa de Servicios: API · Recepción de solicitudes · Respuestas

3. Capa de Lógica de Negocio: Validaciones · Reglas · Procesamiento

4. Capa de Acceso a Datos: Consultas · Inserciones · Actualizaciones · Eliminaciones

1- Capa de Presentación

Propósito: Permitir que los usuarios interactúen con el sistema mediante una interfaz gráfica.

Responsabilidades: Mostrar información, presentar formularios, capturar datos y visualizar los resultados de las operaciones.

Aplicación al proyecto: El joven podrá consultar ofertas laborales, editar su perfil, guardar vacantes y postularse. El administrador tendrá acceso a su panel de control.

Interacción: Envía las acciones del usuario a la capa de Servicios y muestra las respuestas que recibe.

2- Capa de Servicios

Propósito: Establecer un medio de comunicación entre la interfaz y las funcionalidades internas del sistema.

Responsabilidades: Recibir solicitudes, exponer operaciones mediante una API, coordinar su ejecución y devolver respuestas.

Aplicación al proyecto: Se propone crear servicios como UsuariosService, OfertasService y PostulacionesService.

Interacción: Recibe una solicitud desde la capa de Presentación, invoca la lógica correspondiente y devuelve el resultado. Una interfaz de servicios bien definida permite desacoplar la presentación de la lógica interna.

3- Capa de Lógica de Negocio

Propósito: Aplicar las reglas que determinan cómo debe funcionar el sistema.

Responsabilidades: Validar información, comprobar permisos, procesar operaciones y verificar que se cumplan las condiciones necesarias.

Aplicación al proyecto: Antes de registrar una postulación, esta capa verificará que el usuario esté autenticado, que la oferta continúe activa y que no exista una postulación duplicada, según las reglas definidas para la plataforma.

Interacción: Recibe la operación de la capa de Servicios y utiliza la capa de Acceso a Datos para consultar o modificar información. Centralizar estas reglas evita que dependan directamente de los formularios o las consultas a la base de datos.

4- Capa de Acceso a Datos

Propósito: Establecer la comunicación entre la aplicación y la base de datos.

Responsabilidades: Ejecutar las operaciones CRUD, gestionar las consultas y almacenar o recuperar información.

Aplicación al proyecto: Consultará las tablas usuarios, perfiles, ofertas_empleo, postulaciones y las demás entidades del sistema. Interacción: Recibe instrucciones de la lógica de negocio, ejecuta las operaciones necesarias en MySQL y devuelve sus resultados.

La ventaja de esta separación es que las demás capas no necesitan conocer los detalles de las consultas SQL.

Relación entre módulos y capas

Tabla de responsabilidades por módulo

Propuesta de distribución de funciones para la Bolsa de Empleo Local.

Módulo	Presentación	Servicios	Lógica de negocio	Acceso a datos
1. Autenticación	Muestra formularios de inicio de sesión y registro.	Recibe credenciales o respuesta del proveedor social.	Valida identidad y permisos de acceso.	Consulta usuarios y cuentas sociales.
2. Usuarios y roles	Muestra formularios y listado de usuarios.	Expone operaciones de administración de cuentas.	Valida datos y permisos según el rol.	Consulta y modifica usuarios y roles.
3. Perfiles	Permite editar profesión, habilidades y experiencia.	Recibe solicitudes de consulta y actualización.	Valida información y comprueba que el perfil pertenezca al usuario.	Guarda y recupera perfiles.
4. Categorías	Muestra y administra categorías laborales.	Expone operaciones CRUD de categorías.	Verifica permisos y evita categorías duplicadas.	Consulta y modifica categorías.
5. Ofertas laborales	Muestra vacantes, filtros y formularios.	Recibe solicitudes para consultar o gestionar ofertas.	Valida información, estado y permisos de publicación.	Consulta, inserta, actualiza o elimina ofertas.
6. Postulaciones	Presenta el botón Postularme y muestra el estado.	Recibe y procesa solicitudes de postulación.	Verifica usuario, vacante activa y duplicados.	Guarda y consulta postulaciones.
7. Lista de deseos	Muestra ofertas guardadas y permite quitarlas.	Recibe solicitudes para guardar o eliminar favoritos.	Verifica que existan la oferta y el usuario.	Consulta, agrega o elimina favoritos.

8. Historial de registros	Muestra actividades anteriores del usuario.	Solicita y entrega registros históricos.	Autoriza la consulta del historial correspondiente.	Recupera registros de transacciones o actividad.
9. Administración	Presenta el panel y formularios CRUD.	Expone operaciones administrativas.	Comprueba el rol y aplica reglas administrativas.	Modifica las tablas correspondientes.
10. Logs y auditoría	Muestra registros de actividad autorizados.	Recibe eventos y expone consultas de auditoría.	Determina eventos registrables y permisos de consulta.	Almacena y recupera logs.
11. Métricas	Muestra estadísticas y gráficos.	Entrega datos estadísticos al panel.	Define filtros e indicadores del sistema.	Ejecuta consultas agregadas de usuarios, ofertas y postulaciones.

Conclusiones

Como equipo, concluimos que la Arquitectura Orientada a Servicios (SOA) es importante porque permite organizar un sistema en componentes con responsabilidades definidas, facilitando su desarrollo, mantenimiento y crecimiento.

En nuestro proyecto de Bolsa de Empleo Local, separar las funcionalidades en cuatro capas nos ayudará a mantener organizado el código y evitar que toda la aplicación dependa de una sola parte. Por ejemplo, podremos modificar el diseño de la página sin necesidad de cambiar las consultas a la base de datos, o actualizar las reglas de las postulaciones sin modificar toda la plataforma.

Esta separación también facilita el trabajo en equipo, permite reutilizar funcionalidades y hace más sencillo identificar y corregir errores.

Referencias:

Angular. (2025). Architecture Overview. <https://angular.dev/guide/architecture>

Express.js. (s.f.). Express Guide. <https://expressjs.com>

IBM. (2023). Service-Oriented Architecture (SOA). <https://www.ibm.com/topics/soa>

Laravel. (2025). Service Container. <https://laravel.com/docs/12.x/container>

Microsoft Learn. (2024). Introduction to Service-Oriented Architecture. <https://learn.microsoft.com/azure/architecture/>

Oracle. (2017). SOAP Web Services. <https://docs.oracle.com/javaee/7/tutorial/jaxws.htm>

Enlace a github:

<https://github.com/Nicasio01/PSW26B>